

第7章 Servlet技术

- Servlet介绍
- Servlet的编写
- Servlet常用API
- Servlet过滤器
- Servlet监听器

学习目标

- 理解Servlet的基本概念、功能和优点
- 掌握Servlet的编写及配置
- 理解Servlet的生命周期中各个方法的作用，并能够选择合适的Servlet API处理用户的请求
- 掌握什么是Servlet过滤器以及适用场合



第7章 Servlet技术

- Servlet介绍
- Servlet的编写
- Servlet常用API
- Servlet过滤器
- Servlet监听器



Servlet介绍

- Servlet是服务器端的小应用程序，用于响应客户端的请求；并且一般会把处理的结果以HTML的形式返回给客户端。
- Servlet是由服务器端调用和执行的Java类。
- Servlet与JSP有着直接的血缘关系。
(JSP代码就是先转换为Servlet，然后才被编译执行)

Servlet介绍

- Servlet的生命周期

- 当客户端第一次请求Servlet时，Servlet被加载到内存
 - 服务器创建这个Servlet的实例，并调用该对象的init()方法进行初始化
 - 每当客户端发来请求的时候，容器创建请求对象和响应对象，并调用该对象的service()方法对用户的请求进行处理，并对用户进行响应
 - 当服务器端不再需要该Servlet的时候，服务器调用destroy()方法卸载该Servlet
- 注意：在Servlet的生命周期中，service()方法是其中的核心。每当客户端向Servlet发出一个请求时，service()方法就会被调用。



Servlet介绍

- 默认条件下，service()方法会调用与HTTP请求相应的do功能。
 - 客户是以GET方式提交请求，则service()方法会调用doGet()方法；
 - 客户是以POST方式提交请求，则service()方法会调用doPost()方法。
- 注意：用户在地址栏中输入该JSP页面的URL直接访问该页面时，使用的是get请求。



第7章 Servlet技术

- Servlet介绍
- **Servlet的编写**
- Servlet常用API
- Servlet过滤器
- Servlet监听器



Servlet的编写

- 项目1——简单Servlet开发
- **制作一个Servlet的完整过程**
 - 编写servlet
 - 导入需要的类库
 - 继承HttpServlet
 - 重写相应的方法(doGet/doPost或服务)
 - 编译servlet并部署到指定位置
 - 配置servlet
 - 访问servlet



编写Servlet (一)

- 导入需要的类库
- 确定类名并继承HttpServlet

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class FirstServlet extends HttpServlet {}
```



编写Servlet (二)

- 重写doGet和doPost方法

```
public void doGet( HttpServletRequest  
request, HttpServletResponse response )  
throws IOException, ServletException  
{  
    .....  
}
```

(doPost和doGet的头部写法完全一样)



编写Servlet (三)

- 在Servlet中，要使用out对象向客户端输出时，应使用如下方式：

```
PrintWriter out = response.getWriter();  
out.print("<h1>Hello, world!</h1>");
```

- 若要设置显示中文，还需要加入如下语句：
response.setContentType("text/html;charset=
GBK");



编译Servlet并部署

- 编译Servlet需要用到
“...\\tomcat5.5\\common\\lib**servlet-api.jar**”包
(用javac编译时, 需将该包加到classpath环境变量中)
- 编译完成后, 将生成的文件(连同其目录)存放到
“/WEB-INF/classes”下



配置Servlet-方法1

- 需要将Servlet的配置添加到web.xml文件中

```
<web-app version="2.4">
```

Servlet2.4规范

```
<servlet>
```

```
<servlet-name>FirstServlet</servlet-name>
```

```
<servlet-class>servlets.FirstServlet</servlet-class>
```

```
</servlet>
```

类名

```
<servlet-mapping>
```

```
<servlet-name>FirstServlet</servlet-name>
```

```
<url-pattern>/fs</url-pattern>
```

```
</servlet-mapping>
```

```
</web-app>
```

访问该Servlet时用的名字



配置Servlet-方法2

- Servlet3.0规范
- **@WebServlet**(description = "This is my first Servlet", **urlPatterns** = { "/"fs" })
- 重要属性：
 - name , 相当于<servlet-name>
 - **urlPatterns**, 必须, 相当于<url-pattern>
 - value, 同urlPatterns, 二者二选一
 - description, 描述信息
- 可标注于任何一个实现了HttpServlet的类上



访问Servlet

- 做完上述配置以后需要重新启动tomcat
- 然后就可以通过如下方式访问Servlet了
`http://127.0.0.1:8080/ch08/fs`
- 注意：运行前先注释掉其他源码，并clean一下。



第7章 Servlet技术

- Servlet介绍
- Servlet的编写
- **Servlet常用API**
- Servlet过滤器
- Servlet监听器

Servlet常用API

- ServletConfig接口
- ServletContext接口
- **HttpServlet类 (重点)**
- **HttpServletResponse接口 (重点)**
- **HttpServletRequest接口 (重点)**
- RequestDispatcher接口
- 参考: <http://tomcat.apache.org/tomcat-9.0-doc/servletapi/index.html>

Servlet常用API

- ServletConfig接口

- ServletConfig定义了一系列获取配置信息的方法。

- 在Servlet运行期间，经常需要一些配置信息，例如，文件使用的编码等，这些信息都可以在@WebServlet注解的属性中配置。当Tomcat初始化一个Servlet时，会将该Servlet的配置信息封装到一个ServletConfig对象中，通过调用init(ServletConfig config)方法将ServletConfig对象传递给Servlet。

方法说明	功能描述
String getInitParameter(String name)	根据初始化参数名返回对应的初始化参数值
Enumeration getInitParameterNames()	返回一个Enumeration对象，其中包含了所有的初始化参数名
ServletContext getServletContext()	返回一个代表当前Web应用的ServletContext对象
String getServletName()	返回Servlet的名字



Servlet常用API

- ServletContext接口

- 使用ServletContext接口获取Web应用的初始化参数
 - <context-param>元素位于web.xml的根元素<web-app>中，它的子元素<param-name>和<param-value>分别用来指定参数的名字和参数值。可以通过调用ServletContext接口中定义的getInitParameterNames()和getInitParameter(String name)方法，分别获取参数名和参数值。
- 使用ServletContext接口实现多个Servlet对象共享数据
- 使用ServletContext接口读取Web应用下的资源文件

Servlet常用API

• ServletContext接口

- 使用ServletContext接口获取Web应用的初始化参数
- 使用ServletContext接口实现多个Servlet对象**共享数据**
 - 由于一个Web应用中的所有Servlet共享同一个ServletContext对象，所以ServletContext对象的域属性可以被该Web应用中的所有Servlet访问。ServletContext接口中定义了用于增加、删除、设置ServletContext域属性的方法。

方法说明	功能描述
Enumeration getAttributeNames()	返回一个Enumeration对象，该对象包含了所有存放在ServletContext中的所有域属性名
Object getAttribute(String name)	根据参数指定的属性名返回一个与之匹配的域属性值
void removeAttribute(String name)	根据参数指定的域属性名，从ServletContext中删除匹配的域属性
void setAttribute(String name, Object obj)	设置ServletContext的域属性，其中name是域属性名，obj是域属性值



Servlet常用API

• ServletContext接口

- 使用ServletContext接口获取Web应用的初始化参数
- 使用ServletContext接口实现多个Servlet对象共享数据
- 使用ServletContext接口读取Web应用下的资源文件
 - ServletContext接口定义了一些读取Web资源的方法，这些方法是依靠Servlet容器来实现的。Servlet容器根据资源文件相对于Web应用的路径，返回关联资源文件的IO流、资源文件在文件系统的绝对路径等。

方法说明

功能描述

Set getResourcePaths(String path)

返回一个Set集合，集合中包含资源目录中子目录和文件的路径名称。参数path必须以正斜线(/)开始，指定匹配资源的部分路径

String getRealPath(String path)

返回资源文件在服务器文件系统上的真实路径(文件的绝对路径)。参数path代表资源文件的虚拟路径，它应该以正斜线(/)开始，"/"表示当前Web应用的根目录，如果Servlet容器不能将虚拟路径转换为文件系统的真实路径，则返回null

URL getResource(String path)

返回映射到某个资源文件的URL对象。参数path必须以正斜线(/)开始，"/"表示当前Web应用的根目录

InputStream
getResourceAsStream(String path)

返回映射到某个资源文件的InputStream输入流对象。参数path传递规则和getResource()方法完全一致



Servlet常用API

- **HttpServlet类**

- 抽象类，继承自`javax.servlet.GenericServlet`

- `GenericServlet`是[`javax.servlet.Servlet`接口](#)的实现类，没有实现http请求处理；`GenericServlet`的子类`HttpServlet`为http请求中的get、post等方法类型提供了具体操作方法。

- 常用方法

- `protected void doGet`(`HttpServletRequest req`, `HttpServletResponse resp`): 处理GET类型的Http请求
 - `protected void doPost`(`HttpServletRequest req`, `HttpServletResponse resp`): 处理POST类型的Http请求

Servlet常用API

- **HttpServlet类**

- 继承自GenericServlet抽象类的方法:

- `getServletContext()`: 获取ServletContext对象, 即JSP内置对象中的application对象。
- `getServletName()`: 获取Servlet配置时声明在Web应用内部使用的名字。
- `getInitParameter(String name)`: 获取Servlet配置时提供的名为name的参数值



Servlet常用API

- **HttpServletResponse**

- 继承自[javax.servlet.ServletResponse](#)，封装http响应消息。
- 常用方法：
 - 含响应状态行、响应消息头、响应消息体的方法
 - `setContentType(String type)`: 设置响应的内容类型。
 - `setCharacterEncoding(String charset)`: 设置响应的编码字符集为 charset。
 - `getWriter()`: 返回一个PrintWriter对象，利用这个对象可以向客户端输出文本，这个对象的作用类似于JSP的内置对象out。
 - **`sendRedirect(String location)`**: 向客户端发送一个重定向请求，地址为location。用于生成302响应码和Location响应头，从而通知客户端重新访问Location响应头中指定的URL

Servlet常用API

- **HttpServletResponse**

- 中文乱码问题

- 由于计算机中的数据都是以**二进制形式存储**的，所以，当传输文本时，就会发生**字符**和**字节**之间的转换。字符与字节之间的转换是通过查码表完成的，将字符转换成字节的过程称为**编码**，将字节转换成字符的过程称为**解码**，如果**编码和解码使用的码表不一致**，就会导致乱码问题。



Servlet常用API

- HttpServletResponse

- 输出页面的中文输出乱码问题解决方案:

- 第一种

- //设置HttpServletResponse使用utf-8编码
- response.setCharacterEncoding("utf-8");
- //通知浏览器使用utf-8解码
- response.setHeader("Content-Type ",
"text/html;charset=utf-8");

- 第二种

- //包含第一种方式的两个功能
- **response.setContentType("text/html;charset=utf-8");**



Servlet常用API

- HttpServletResponse

- 网页定时刷新并跳转:

- `response.setHeader( "Refresh" , " 2; url=https://www.baidu.com" );`

- 网页自动刷新:

- `response.setHeader( "Refresh" , " 2" );`

- 禁止浏览器缓存页面

- `response.setDateHeader( "Expires" ,0);`
- `response.setHeader( "Cache-Control" ,," no-cache" );`
- `response.setDateHeader( "Pragma" ,," no-cache" );`



Servlet常用API

- **HttpServletRequest接口**

- 继承自[javax.servlet.ServletRequest](#)，封装http请求消息
- HttpServletRequest常用方法
 - 含请求行、请求头、请求转发、请求参数、请求属性等方法
 - `getRequestURL()`: 获取请求的URL地址，包括协议名，服务器名，端口号和所请求服务的路径，但不包含请求时所带的参数。
 - `getRequestURI()`: 获取请求行中资源名称部分，即主机和端口后，参数之前部分。
 - `getContextPath()`: 获取Web应用的根路径。
 - `getServletPath()`: 获取Servlet的访问地址。



Servlet常用API

- HttpServletRequest的方法输出

调用方法	输出内容
getRequestURL()	http://localhost:8080/ch08/fs
getRequestURI()	/ch08/fs
getContextPath()	/ch08
getServletPath()	/fs

Servlet常用API

– HttpServletRequest常用方法

- **getParameter(String name)**: 获得名为name的参数的单个值。
- **getParameterValues(String name)**: 获得名为name的参数的多个值。
- **getAttribute(String name)**: 获得名为name的属性值。
- **setAttribute(String name,String value)**: 设置名为name的属性值为value。
- **getSession()**: 获取session对象。
- **getRequestDispatcher(String path)**: 获取请求转发对象, 转向地址为path。所获得的RequestDispatcher对象的forward()方法实现真正的跳转。



Servlet常用API

– HttpServletRequest常用方法

• 表单请求参数的中文乱码问题

– 原因：浏览器在传递请求参数时，采用默认本地编码方式，但在解码时采用默认的ISO8859-1

– 解决方案：

» post方式提交表单参数乱码

» 对表单参数进行处理时，设置请求体的字符编码为表单页面的编码，如：

```
request.setCharacterEncoding("utf-8");
```

» 该方式对get方式无效

» get方式提交表单参数乱码

» 先用默认编码表ISO8859-1将中文参数重新编码，再使用表单页所用中文码表（gbk，utf-8等）进行解码，如：`name=new String(name.getBytes("iso8859-1"), "utf-8")`；

» 可配置Tomcat解决get方式提交参数乱码（不建议）



Servlet常用API

- RequestDispatcher 接口
 - RequestDispatcher接口常用方法
 - forward (ServletRequest request, ServletResponse response)
 - 将请求从一个Servlet传递给另外的一个Web资源；其他Web资源处理完请求后，直接将响应结果返回到客户端。
 - 不仅可以实现请求转发，还可以使转发页面和转发到的页面共享数据。
 - include (ServletRequest request, ServletResponse response)
 - 将其他的资源作为当前响应内容包含进来
 - 将Servlet请求转发给其他Web资源进行处理
 - 不同于请求转发，在请求包含返回的响应消息中，既包含当前Servlet的响应消息，也包含其他Web资源所做出的响应消息
 - 用include()方法实现请求包含后，浏览器的URL地址不变。



Servlet常用API

• 项目2——模拟登录

- 实现用户的模拟登录行为，用户在登录页面选择身份（管理员和普通用户）提交后，请求由一个Servlet处理，Servlet根据用户身份将请求转发到不同的欢迎页面。
- 登录页面login.html，该页面提供用户身份选择的单选按钮以及登录按钮；处理用户请求的LoginServlet.java，它是一个Servlet，获取用户输入的身份，根据身份将请求转发到管理员欢迎页面admin.html或普通用户欢迎页面common.html。
- 注意：
 - servlet编写及配置
 - HttpServletRequest接口的重要方法 --请求参数获取、session属性操作、请求转发
 - HttpServletResponse接口的重要方法--重定向

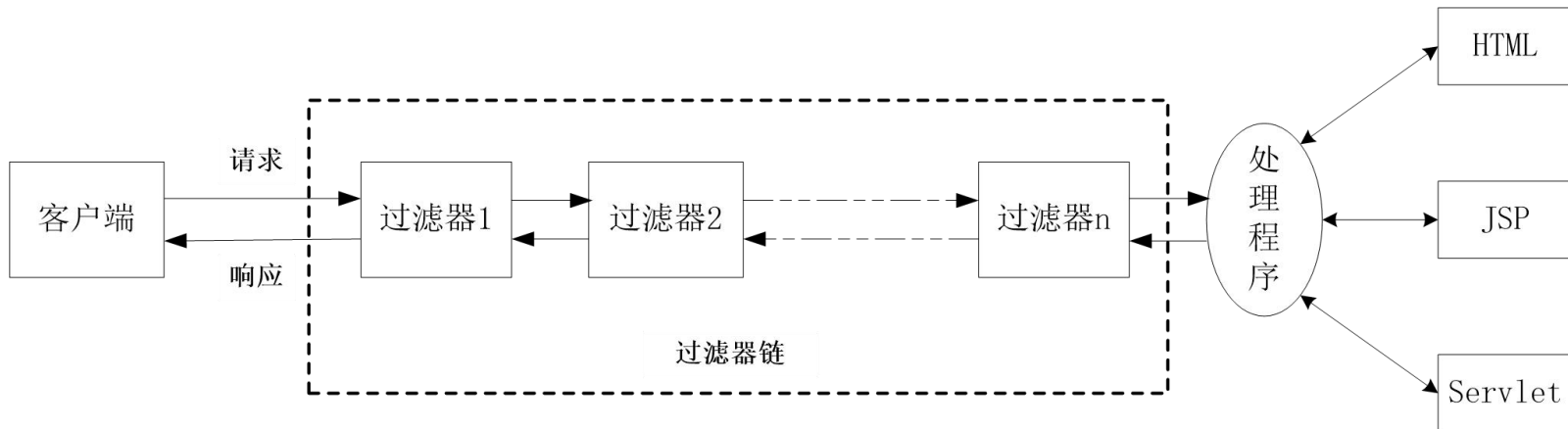


第7章 Servlet技术

- Servlet介绍
- Servlet的编写
- Servlet常用API
- **Servlet过滤器**
- Servlet监听器

Servlet过滤器

- 是一种Java组件，位于客户端和处理程序之间，可以对请求和响应进行检查和修改，通常用来完成通用操作，如：字符编码，安全控制等。
- 工作原理





Servlet过滤器API

- Filter接口
- FilterChain接口
- FilterConfig接口



Servlet过滤器API

- Filter接口
 - 编写过滤器必须实现的接口，包含三个方法：
 - `init(FilterConfig filterConfig)` 初始化方法
 - `doFilter(ServletRequest request, ServletResponse response, FilterChain chain)` 完成实际的过滤操作
 - `destroy()` 释放被Filter对象打开的资源
- FilterChain接口
- FilterConfig接口



Servlet过滤器API

- Filter接口
- FilterChain接口
 - **doFilter**(ServletRequest request, ServletResponse response) 用于调用Filter链中的下一个过滤器，若为最后一个，则将请求提交给处理程序或将响应发送给客户端
- FilterConfig接口



Servlet过滤器API

- Filter接口
- FilterChain接口
- FilterConfig接口
 - 用于封装Filter的配置信息
 - `getFilterNames()`
 - `getServletContext()`
 - `getInitParameter(String name)`
 - `getInitParameterNames()`



Servlet过滤器

- 项目3-不缓存页面的过滤器
- 注意：
 - 1.过滤器必须实现Filter接口
 - 2.过滤器过滤规则的配置方式
 - 3.禁止缓存的实现代码
 - 4.过滤器和URL之间的关系



Servlet过滤器

- 项目3-不缓存页面的过滤器
- 注意：
 - **1.过滤器必须实现Filter接口**
 - 重写doFilter(req, res, chain)方法
 - **2.过滤器过滤规则的配置方式**
 - Servlet3.0 规范利用
`@WebFilter(description=" ", urlPatterns={ "/"* })`
 - 早期Servlet版本,需要在web.xml中配置<filter>及<filter-mapping>元素
 - **3.禁止缓存**的实现代码
 - **4.过滤器和URL之间的关系**



Servlet过滤器

- 项目3-不缓存页面的过滤器
- 注意：
 - 1.过滤器必须实现Filter接口
 - 2.过滤器过滤规则的配置方式
 - 3.禁止缓存的核心实现代码
 - `response.setHeader( "Pragma" , " no-cache" ) ;`
 - `response.setHeader( "Cache-Control" , " no-cache" ) ;`
 - `response.setHeader( "Expires" , " -1" ) ;`
 - 4.过滤器和URL之间的关系



Servlet过滤器

- 项目3-不缓存页面的过滤器
- 注意：
 - 1.过滤器必须实现Filter接口
 - 2.过滤器过滤规则的配置方式
 - 3.禁止缓存的实现代码
 - 4.过滤器和URL之间的关系
 - 一个过滤器可以配置多个映射，即指定多个URL过滤规则
 - 一个URL可以对应多个Servlet过滤器，这些过滤器根据在配置文件中出现的先后顺序组成一个过滤器链



Servlet过滤器

- 项目4-登录验证过滤器
- 注意：
 - 过滤规则的配置
 - `@WebFilter(description=" " ,urlPatterns={ "/admin.html" ,
/common.html" })`
 - session属性操作
 - 服务器重定向



@WebFilter注解的常用属性

属性名	类型	描述
filterName	String	指定过滤器的名称。默认是过滤器类的名称。
urlPatterns	String[]	指定一组过滤器的URL匹配模式。
value	String[]	该属性等价于urlPatterns属性。urlPatterns和value属性不能同时使用。
servletNames	String[]	指定过滤器将应用于哪些Servlet。取值是@WebServlet 中的 name 属性的取值。
dispatcherTypes	DispatcherType	指定过滤器的转发模式。具体取值包括：ERROR、FORWARD、INCLUDE、REQUEST。
initParams	WebInitParam[]	指定过滤器的一组初始化参数。



@WebFilter注解拦截不同访问方式的请求

- @WebFilter注解有一个特殊的属性dispatcherTypes，它可以指定过滤器的转发模式，dispatcherTypes属性有4个常用值，**REQUEST**、**INCLUDE**、**FORWARD**和**ERROR**。
 - **REQUEST**:如果用户通过RequestDispatcher对象的include()方法或forward()方法访问目标资源，那么过滤器不会被调用。除此之外，该过滤器会被调用。
 - **INCLUDE**:如果用户通过RequestDispatcher对象的include()方法访问目标资源，那么过滤器将被调用。除此之外，该过滤器不会被调用。
 - **FORWARD**:如果通过RequestDispatcher对象的forward()方法访问目标资源，那么过滤器将被调用。除此之外，该过滤器不会被调用。
 - **ERROR**:如果通过声明式异常处理机制调用目标资源，那么过滤器将被调用。除此之外，过滤器不会被调用。



第7章 Servlet技术

- Servlet介绍
- Servlet的编写
- Servlet常用API
- Servlet过滤器
- Servlet监听器-了解

Servlet监听器

- 与Java中的监听器类似，Servlet监听器也用来监听重要事件的发生，只不过它监听的对象是Web组件。
- Servlet共提供了8个监听器接口和6个事件类，分别实现了对Servlet上下文、HTTP会话和客户端请求的监听。

Listener接口	Event类
ServletContextListener	ServletContextEvent
ServletContextAttributeListener	ServletContextAttributeEvent
HttpSessionListener	HttpSessionEvent
HttpSessionActivationListener	
HttpSessionAttributeListener	HttpSessionBindingEvent
HttpSessionBindingListener	
ServletRequestListener	ServletRequestEvent
ServletRequestAttributeListener	ServletRequestAttributeEvent

Servlet监听器

- Servlet共提供了8个监听器接口

类型	描述
ServletContextListener	用于监听ServletContext对象的创建与销毁过程
HttpSessionListener	用于监听HttpSession对象的创建和销毁过程
ServletRequestListener	用于监听ServletRequest对象的创建和销毁过程
ServletContextAttributeListener	用于监听ServletContext对象中的属性变更
HttpSessionAttributeListener	用于监听HttpSession对象中的属性变更
ServletRequestAttributeListener	用于监听ServletRequest对象中的属性变更
HttpSessionBindingListener	用于监听JavaBean对象绑定到HttpSession对象和从HttpSession对象解绑的事件
HttpSessionActivationListener	用于监听HttpSession中对象活化和钝化的过程

小结

- Servlet是用Java语言编写的服务器端程序，它可以处理客户端发送的请求并返回一个响应。
- **Servlet生命周期中的重要方法**包括init()、service()和destroy()，其中**service()方法**是通过调用doGet()或doPost()方法发挥作用的。
- Servlet编写完成后，必须正确配置才能使用。
- **Servlet过滤器**的使用在Web开发中也较为常见，比较典型的应用包括统一字符编码、字符压缩、加密，实施安全控制等等。
- Servlet监听器可以对Servlet上下文、HTTP会话和客户端请求进行监听。

作业

- 1. 什么是Servlet? 其生命周期中包含哪些重要的方法?
- 2. 比较JSP和Servlet的相同点和不同点。
- 3. 如何在Servlet中实现页面跳转?
- 4. 什么是Servlet过滤器? 它的工作原理是什么?
- 5. 如何编写一个Servlet? 如何部署和配置?
- 6. 简介Servlet常用API。